

Chapter 1

Microcontroller Operation & Architecture

ENEL712 Embedded Systems Design — Lecture 2

Summary

How an MCU executes instructions: the fetch-decode-execute cycle, core CPU components, Harvard architecture, pipelining, and the RISC vs CISC trade-off.

Topics Covered

- Microcontroller overview & integrated components
- Fetch–Decode–Execute cycle
- Core CPU components
- Architecture and performance concepts
- RISC vs CISC comparison

1.1 Overview of a Microcontroller

A microcontroller (MCU) is a complete computer on a single IC, integrating the CPU core, memory (RAM, ROM, EEPROM), ADCs, timers, and I/O peripherals.

- Integration reduces external components and simplifies the power supply (single 3.3 V / 5 V rail).
- Enables deterministic real-time behaviour and low-power modes.

1.2 Microcontroller Operation: Fetch–Decode–Execute

- Power-Up: components disabled while the supply stabilises and the oscillator settles (milliseconds).
- Initialization: reset releases, CPU starts at the reset vector, firmware initialises peripherals and SFRs.

- Execution Cycle: PC starts at zero; instructions are fetched, decoded, executed; PC increments and the cycle repeats millions/billions of times per second.

1.3 Core CPU Components

Component	Role
Program Counter (PC)	Holds address of next instruction; increments after fetch, jumps on branches/calls/interrupts.
Instruction Register (IR)	Temporary register holding the fetched instruction during decode and execute.
Instruction Decoder (ID)	Translates instruction bits into control signals (registers, ALU operation).
Arithmetic Logic Unit (ALU)	Performs integer arithmetic (add/sub/multiply) and logic (shift/rotate) operations.
General Purpose Registers (GPRs)	32 fast 8-bit registers giving the ALU single-cycle access.
Status Register (SREG)	Flags (zero, carry, negative, overflow) and Global Interrupt Enable for conditional branches.

1.4 Architecture and Performance Concepts

1.4.1 Harvard Architecture

Used by the AT90 AVR. Separates program and data memory with dedicated buses, enabling simultaneous instruction fetch and data access.

1.4.2 Pipelining

- Two-stage AVR pipeline overlaps fetch and execute so most instructions complete every clock cycle.
- Branches invalidate the prefetched instruction, forcing a flush and refill that costs extra cycles.

1.5 RISC vs. CISC Comparison

Feature	RISC	CISC
Example	AVR, ARM	x86, 8051
Instruction set	Simple, uniform, mostly single-cycle	Complex, varied, often multi-cycle
Pipelining	Highly effective due to uniform timing	Harder; requires complex hardware
Power / hardware	Lower power, simpler hardware	Complex hardware, can have denser code

1.6 Use Cases & Applications

- Boot-from-reset behaviour: every embedded product walks through the same Power-Up → Initialization → Execution sequence on reset; understanding it helps debug 'why does my MCU appear stuck for 50 ms after power-on?' (oscillator stabilisation).
- Performance estimation: knowing the AVR is single-cycle per instruction lets you back-of-the-envelope timing for ISRs and tight loops in cycles, not seconds.

- Driver writing: SFRs are how you talk to peripherals — every write to a UART/SPI/I²C register is a side-effect on a memory-mapped SFR.
- Conditional code in C compiles to flag-based branches (zero, carry, overflow) read out of SREG — useful when reading disassembly.

1.7 Worked Example: Trace of a Simple Instruction

Consider ADD R16, R17 on an AT90 (8-bit AVR). Walk through one fetch–decode–execute cycle.

- Fetch: PC points at the instruction in Flash; the 16-bit opcode is loaded into the IR; the next instruction is prefetched (pipeline stage 1).
- Decode: ID interprets the opcode as 'ALU add, source R17, destination R16'.
- Execute: ALU reads R16 and R17, sums them, writes the result back to R16, and sets SREG flags (Z if zero, C if carry-out, N if MSB set).
- PC increments to the next instruction; total elapsed time at 16 MHz $\approx$ 62.5 ns.

1.8 Practical Notes

- Why startup takes time: the crystal oscillator typically needs several thousand cycles to stabilise — datasheets specify this as 'startup time from reset' (often 65 ms on AVRs).
- Why some 'simple' C statements compile to many instructions: 16- or 32-bit operations on an 8-bit ALU are decomposed into multiple 8-bit ADDs/SUBs with carry propagation through SREG.
- When SREG matters in C: the volatile keyword and the cli()/sei() pair around shared variables — they suppress interrupts so SREG.I doesn't change underneath you.

Key Definitions

Microcontroller (MCU)

Single IC integrating CPU, memory, and peripherals (timers, comm interfaces, ADC, GPIO).

Special Function Registers (SFRs)

Registers used to control and check the status of on-chip peripherals and I/O ports.

Harvard Architecture

Separate storage and buses for program and data memory.

Pipelining

Overlapping instruction fetch/decode/execute stages to increase throughput.

Program Counter (PC)

Register holding the address of the next instruction to fetch.

Instruction Register (IR)

Register holding a fetched instruction while it is being decoded.

Arithmetic Logic Unit (ALU)

CPU component executing arithmetic and logic operations.

Status Register (SREG)

Flag register storing results of the most recent operation (zero, carry, overflow, etc.).

Reset Vector

Fixed Flash address where the CPU starts executing after reset; usually 0x0000 on AVR.

Memory-Mapped Register

Hardware register that lives at a normal data address so it can be accessed with regular load/store instructions.

Cycle Budget

Number of clock cycles available for a periodic task; ISR length must be $< (\text{period} \times \text{clock frequency})$.

Sources

- Lecture 2.pdf